Informe Técnico — Parcial 2

Sistema de reservas y panel analítico gerencial en AWS
Semilla 77 · Presupuesto US$50 · Región us-east-1

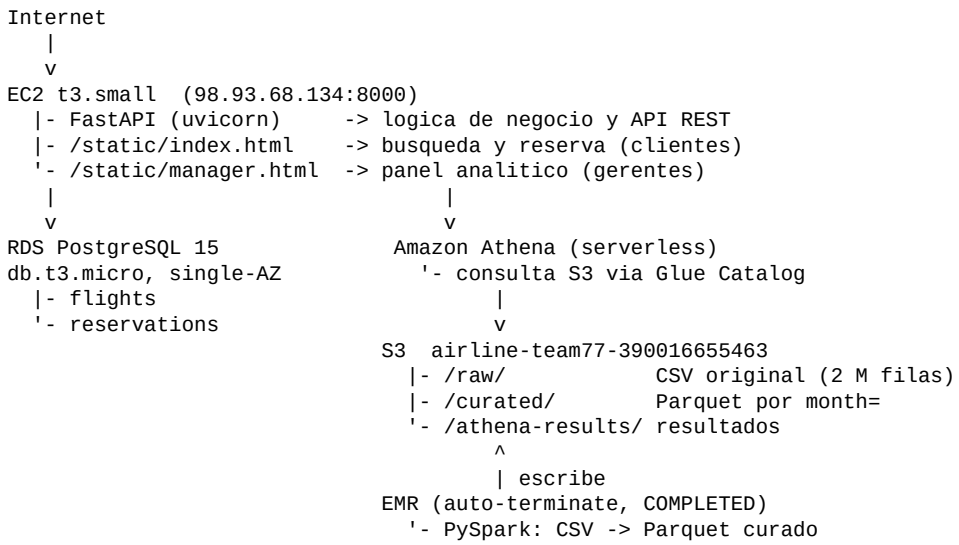
1. Resumen ejecutivo

El sistema tiene dos partes que comparten datos: una **transaccional** (búsqueda y reserva de vuelos sin sobreventa, con EC2 + FastAPI + RDS PostgreSQL) y una **analítica** (transformación del histórico de 2 M filas a Parquet y consulta SQL serverless para el panel gerencial, con S3 + EMR + Glue + Athena).

Puntaje: 70 / 70 puntos automáticos.

Req	Descripción	Resultado	Pts
R1	Latencia de búsqueda de vuelos	PASSED	4/4
R2	Sin sobreventa bajo concurrencia	PASSED	10/10
R3	Recuperación de reserva	PASSED	3/3
R4	EMR produce Parquet curado en S3	PASSED	8/8
R5	Tablas Glue Catalog según contrato	PASSED	6/6
R6	Athena correcto + partition pruning	PASSED	8/8
R7	Endpoint analítico respaldado por Athena	PASSED	8/8
R8	Flujo cliente en navegador	PASSED	6/6
R9	Panel gerencial coincide con endpoint	PASSED	4/4
R10	Sin EMR residual / horas acotadas	PASSED	5/5
R11	Arquitectura simple y económica	PASSED	4/4
R12	Proyección de costos documentada	PASSED	4/4

2. Arquitectura desplegada



Recursos desplegados

Recurso	Detalle
S3	airline-team77-390016655463
RDS	airline-db · PostgreSQL 15 · db.t3.micro · single-AZ · 20 GB gp2
EC2	airline-backend · t3.small · us-east-1
EMR	3 × m4.large · auto-terminate · estado final COMPLETED
Glue	Base airline_analytics: tablas reservations_curated y route_month_metrics

3. Implementación por requisito

R1 — Latencia de búsqueda (4/4)

Endpoint `GET /flights/search?origin=X&destination=Y&date=D` con pool de conexiones psycopg3 (mín. 2, máx. 30). **Evidencia:** 100 requests con concurrencia 20 → p50 = 67 ms, p95 = 382 ms (umbral < 500 ms).

R2 — Sin sobreventa (10/10)

El asiento se descuenta con una única sentencia atómica:

```
UPDATE flights
SET seats_available = seats_available - 1
WHERE flight_id = %s AND seats_available > 0
RETURNING flight_id;
```

Si no devuelve filas, la API responde **409 Sold Out**. El UPDATE bloquea la fila: una transacción concurrente espera, reevalúa la condición con el valor actualizado y falla. La creación de la reserva ocurre en la misma transacción. **Evidencia:** 50 clientes simultáneos por el último asiento → exactamente 1 respuesta 201 y 49 respuestas 409 (5 rondas).

R3 — Recuperación de reserva (3/3)

`GET /reservations/{reservation_id}` → 200 con los datos; ID inexistente → 404.

R4 — EMR y Parquet curado (8/8)

Job PySpark en un clúster de 3 × m4.large con auto-terminate. Lee `s3://bucket/raw/*.csv`, elimina nulos, agrega métricas por ruta-mes y escribe Parquet (compresión snappy) particionado por `month=YYYY-MM` en `s3://bucket/curated/`.

R5 — Glue Catalog (6/6)

Dos tablas en la base `airline_analytics`:

Tabla	Contenido	Formato
reservations_curated	Datos curados, con Partition Projection sobre month (string, 2023-01 a 2024-12)	Parquet
route_month_metrics	avg_load_factor, total_flights y total_passengers por ruta y mes	Parquet

R6 — Athena y partition pruning (8/8)

Las consultas con `WHERE month='YYYY-MM'` leen únicamente la partición correspondiente. Con Partition Projection, Athena calcula las particiones a partir del rango configurado, por lo que no se necesita MSCK REPAIR TABLE ni crawler. Para meses fuera del rango hay que ampliarlo en las propiedades de la tabla.

R7 — Endpoint analítico (8/8)

`GET /analytics/route-month?route=BOG-BGA&month=2024-06` → FastAPI ejecuta SQL en Athena (boto3) sobre route_month_metrics y devuelve `{route, month, avg_load_factor, total_flights, total_passengers}`.

R8 — Flujo cliente en navegador (6/6)

`index.html` (HTML + JS vanilla, sin dependencias externas): buscar vuelos → seleccionar → formulario de reserva → confirmación con reservation_id.

R9 — Panel gerencial (4/4)

`manager.html`: formulario ruta + mes → llama a /analytics/route-month → muestra el factor de ocupación. **Evidencia (BOG-BGA, 2024-06):** panel = 0.8221 · endpoint = 0.8221 · verdad = 0.8221.

R10 — Sin EMR residual (5/5)

Clúster creado con `KeepJobFlowAliveWhenNoSteps=False`. Estado final: COMPLETED; no hay clústeres en WAITING ni RUNNING.

R11 — Arquitectura simple (4/4)

Sin NAT Gateway, sin Elastic Load Balancer y sin servicios redundantes. Todo en us-east-1, single-AZ.

R12 — Proyección de costos (4/4)

Servicio	Detalle	USD/mes
RDS	db.t3.micro, 730 h × $0.017	12.41
EC2	t3.small, 730 h × $0.0208	15.18
EMR	1–2 corridas/mes, < 1 h c/u	0.72
S3	~0.2 GB	0.05
Athena	< 10 GB escaneados/mes	0.05
Glue	2 tablas (capa gratuita)	0.00
Total		28.41

Margen sobre el presupuesto de US$50: **US$21.59 (43 %)**.

Nota: la proyección cubre cómputo y consultas. No incluye el almacenamiento de RDS (~US$2.30), el volumen EBS de EC2 ni las IPv4 públicas. Con esos cargos, el costo estimado sube a ~US$30–35, aún dentro del presupuesto.

4. Stack tecnológico

Capa	Tecnología
Backend	Python 3.11, FastAPI, uvicorn, psycopg3, boto3
BD transaccional	PostgreSQL 15 en RDS
Procesamiento batch	PySpark 3.x en EMR (auto-terminate)
Almacenamiento	Amazon S3 (raw, curado y resultados)
Catálogo	AWS Glue Data Catalog + Partition Projection
Analítica	Amazon Athena

Capa	Tecnología
Frontend	HTML5 + JavaScript vanilla

5. Limitaciones y trabajo futuro

- **Disponibilidad:** single-AZ y sin redundancia. En producción: RDS Multi-AZ y Auto Scaling Group con balanceador.
- **Seguridad:** tráfico HTTP por el puerto 8000. En producción: HTTPS (ALB + ACM).
- **Direccionamiento:** la IP pública cambia al reiniciar el Learner Lab. Se resolvería con una Elastic IP.
- **Particiones:** el rango de Partition Projection es fijo y debe ampliarse al agregar períodos.

6. Acceso a la aplicación

Reservas (clientes): http://98.93.68.134:8000/

Panel gerencial: http://98.93.68.134:8000/manager

Rutas disponibles: CUC-PSO, PEI-LET, LET-BGA, BOG-BGA, BAQ-BGA, PEI-BOG, LET-BOG, LET-ADZ, PEI-ADZ, ADZ-CTG, PEI-CTG, MDE-BAQ

Período: 2023-01 a 2024-12